

Taller — Complejidad algorítmica y Arreglos

SD1004 · Estructura de Datos — Institución Universitaria Pascual Bravo Docente:
Juan Duque · Semestre 2026-II Cubre: Clase 2 (Big O) · Clase 3 (Arreglos 1D) · Clase 4
(Arreglos 2D)

Instrucciones generales

- Responde directamente en este documento (edítalo en Word, Markdown o entrégalo en PDF).
 - No necesitas escribir código en ningún lenguaje: usa pseudocódigo, dibujos o explicaciones en tus propias palabras. Lo que se evalúa es el razonamiento, no la sintaxis.
 - Donde se pida un dibujo (arreglos, memoria), puedes hacerlo a mano y adjuntar una foto, o usar draw.io.
 - Justifica siempre tu respuesta — una respuesta correcta sin justificación no cuenta completa.
 - Fecha y forma de entrega: la indicada en el aula/repositorio del curso.
-

Parte 1 — Complejidad algorítmica (Big O)

1.1 Clasifica la complejidad

Para cada situación, indica si corresponde a $O(1)$, $O(\log n)$, $O(n)$ o $O(n^2)$, y explica por qué usando tus propias palabras (no hace falta fórmula matemática).

1. Buscar una palabra en un diccionario físico, abriendo siempre por la mitad de las páginas que quedan.

R// $O(\log n)$: Al dividir las páginas a la mitad en cada paso, descartas la mitad de los elementos restantes de una sola vez.

2. Revisar, uno por uno, cada carné en una caja de carnés de estudiantes hasta encontrar el que buscas.

R// $O(n)$: En el peor de los casos debes revisar todos los n carnés uno tras otro. Si se duplican los carnés, el tiempo se duplica.

3. Sacar la primera carta de un mazo ya barajado.

R// $O(1)$: Es un acceso directo al primer elemento; tomar la carta superior siempre toma exactamente el mismo tiempo sin importar si el mazo tiene 10 o 10,000 cartas.

4. Comparar cada estudiante de un salón con cada uno de los demás estudiantes, para ver si algún par cumple años el mismo día.

R// $O(n^2)$: Para cada uno de los n estudiantes, debes recorrer nuevamente a los otros n estudiantes. Es un proceso de parejas ($n \times n$).

5. Consultar la hora en tu reloj.

R// **O(1)**: Solo miras el reloj una vez; no depende de ninguna cantidad de datos.

1.2 De la vida real al análisis

Piensa en una tarea cotidiana (que no sea de las ya usadas en clase) que hagas de forma $O(n)$ — es decir, que si la cantidad de elementos se duplica, el tiempo que tardas también se duplica.

- Descríbela en 2-3 líneas.
- Explica qué pasaría si la "entrada" (n) se hiciera 10 veces más grande.

R// Lavar una fila de platos sucios a mano uno por uno. Si hay n platos, tomo cada plato, lo enjabono, lo enjuago y lo pongo a secar.

Si en lugar de 5 platos tengo 50 platos (10 veces más), me tomará exactamente 10 veces más tiempo lavar toda la pila.

1.3 ¿Cuál escalar mejor?

Tienes dos formas de resolver el mismo problema:

- Forma A: $O(n^2)$
- Forma B: $O(n \log n)$

Para un conjunto pequeño de datos ($n = 5$), la Forma A es más rápida en la práctica. Aun así, ¿cuál recomendarías usar si no sabes qué tan grande será n en el futuro? Justifica tu respuesta pensando en qué pasa a medida que n crece.

R// **Forma B: $O(n \log n)$** : Aunque para un n muy pequeño ($n=5$) la Forma A pueda ser más rápida por diferentes cosas, a medida que n crece a miles o millones de datos, $O(n^2)$ crece de forma cuadrática explosiva (ej., $1,000^2 = 1,000,000$ operaciones), mientras que $O(n \log n)$ crece mucho más despacio (ej., $1,000 \times 10 = 10,000$ operaciones). La Forma B escala infinitamente mejor para volúmenes de datos inciertos o grandes.

1.4 Verdadero o falso (justifica siempre)

- a. Un algoritmo $O(1)$ siempre es más rápido en segundos reales que uno $O(n)$.

R// **Falso**: $O(1)$ significa que el tiempo es constante según el tamaño de la entrada, pero la operación constante podría ser muy pesada (ej. *esperar 10 segundos*) mientras que una operación $O(n)$ para $n=2$ podría tardar 2 milisegundos.

- b. La notación Big O describe cómo crece el tiempo de ejecución a medida que crecen los datos, no el tiempo exacto en segundos.

R// **Verdadero**: Big O mide la tendencia de crecimiento del algoritmo respecto al volumen de datos (n), no los segundos o milisegundos exactos del reloj.

c. Buscar por índice en un arreglo (arreglo[5]) es $O(n)$, porque hay que recorrer los primeros 5 elementos.

R// Falso: Acceder por índice *arreglo[5]* es $O(1)$ porque la computadora calcula matemáticamente la posición exacta en memoria en un solo paso, sin tener que recorrer los índices 0, 1, 2, 3, 4.

Parte 2 — Arreglos 1D

2.1 Diseña el arreglo

Vas a guardar las calificaciones de 6 estudiantes de un curso en un arreglo llamado *notas*.

1. Dibuja el arreglo con sus 6 casillas, mostrando el índice de cada una (recuerda: empieza en 0).

VALOR	3,5	4,2	2,8	5,0	3,9	4,5
INDICE	[0]	[1]	[2]	[3]	[4]	[5]

2. Si *notas* = [3.5, 4.2, 2.8, 5.0, 3.9, 4.5], ¿qué valor y qué índice tiene la tercera nota que ingresaste?

R// Su valor es **2.8** y se encuentra en el índice **[2]**.

3. ¿Cuál es la instrucción (en pseudocódigo) para acceder directamente a la nota del último estudiante sin recorrer el arreglo?

R// *nota_ultimo* = *notas*[5]

2.2 Direcciones de memoria

Supón que el arreglo *notas* del punto anterior se guarda en memoria RAM empezando en la dirección 0x2000, y cada valor ocupa 4 bytes.

1. Calcula la dirección de memoria de *notas*[0], *notas*[3] y *notas*[5], usando la fórmula:
dirección = base + (índice × tamaño)

R// ***notas*[0]:** $0x2000 + (0 \times 4) = 0x2000$

***notas*[3]:** $0x2000 + (3 \times 4) = 0x2000 + 12 \text{ bytes} = 0x200C$

***notas*[5]:** $0x2000 + (5 \times 4) = 0x2000 + 20 \text{ bytes} = 0x2014$

2. Explica en tus palabras por qué acceder a *notas*[3] es igual de rápido que acceder a *notas*[0] — no importa cuál pidas, es $O(1)$.

R// Acceder a *notas*[3] tarda lo mismo que a *notas*[0] porque la computadora no recorre casilla por casilla. Simplemente aplica la fórmula matemática de multiplicación y suma en un solo ciclo de procesador para ir directamente a esa dirección de memoria RAM.

2.3 Búsqueda lineal vs. binaria

Tienes el arreglo ordenado edades = [15, 18, 20, 23, 27, 31, 35, 40].

1. Simula paso a paso una búsqueda lineal para encontrar el valor 31. ¿Cuántas comparaciones hiciste?

R// **Paso 1:** Compara con índice 0 (15). No es.

Paso 2: Compara con índice 1 (18). No es.

Paso 3: Compara con índice 2 (20). No es.

Paso 4: Compara con índice 3 (23). No es.

Paso 5: Compara con índice 4 (27). No es.

Paso 6: Compara con índice 5 (31). **Encontrado.**

6 comparaciones

2. Simula paso a paso una búsqueda binaria para encontrar el mismo valor 31. ¿Cuántas comparaciones hiciste?

R// **Paso 1:** Centro entre índice 0 y 7 es $(0+7)/2 = 3.5$, índice 3 (23). Compara: $31 > 23$, buscamos en la mitad derecha (índices 4 a 7).

Paso 2: Centro entre 4 y 7 es $(4+7)/2 = 5.5$, índice 5 (31). Compara: **Encontrado.**

2 comparaciones.

3. ¿Por qué la búsqueda binaria sólo funciona si el arreglo ya está ordenado?

R// Porque la búsqueda binaria se basa en descartar la mitad superior o inferior según si el valor buscado es mayor o menor que el elemento que está en el centro. Si los datos están desordenados, no se puede saber en cuál mitad se encuentra el número.

4. Si el arreglo tuviera 1 millón de elementos, ¿cuál de las dos búsquedas seguirías usando? Justifica con lo visto sobre $O(n)$ vs. $O(\log n)$.

R// Utilizaría la **Búsqueda Binaria**. La búsqueda lineal $O(n)$ podría requerir hasta 1,000,000 de comparaciones, mientras que la búsqueda binaria $O(\log n)$ requerirá como máximo *aprox 20 comparaciones* ($\log_2(1,000,000)$ *aprox 20*).

2.4 Insertar un elemento

Tienes edades = [15, 18, 20, 23, 27] (5 casillas, sin espacio libre).

1. ¿Qué tan costoso es insertar un nuevo valor al final, si hay espacio disponible? ¿Y si no hay espacio y hay que crear un arreglo más grande?

R// **Sí hay espacio:** Es muy económico $O(1)$, solo se coloca el valor en el índice disponible.

Si no hay espacio: Es costoso $O(n)$, hay que crear un arreglo nuevo más grande y copiar todos los elementos existentes uno por uno al nuevo arreglo.

2. ¿Qué tan costoso es insertar el valor 21 en la mitad (para que quede ordenado)?
Explica qué hay que hacer con los demás elementos.

R//Es costoso $O(n)$. Se deben desplazar hacia la derecha todos los elementos mayores que 21 (el 23 y el 27) para abrir el espacio donde irá el 21.

3. Compara ambos casos usando notación Big O.

R// Insertar al final con espacio libre: $O(1)$.
Insertar en la mitad o redimensionar arreglo: $O(n)$.

Parte 3 — Arreglos 2D

3.1 Diseña la matriz

Vas a guardar la disposición de un salón de clase de 3 filas x 4 columnas, donde cada casilla indica si el puesto está ocupado (1) o libre (0).

1. Dibuja la matriz salón con sus 3 filas y 4 columnas, con los índices [fila][columna] de cada casilla.

FILA / COLUMNA	COLUMNA 0	COLUMNA 1	COOLUMNA 2	COLUMNA 3
FILA 0	salón[0][0]	salón[0][1]	salón[0][2]	salón[0][3]
FILA 1	salón[1][0]	salón[1][1]	salón[1][2]	salón[1][3]
FILA 2	salón[2][0]	salón[2][1]	salón[2][2]	salón[2][3]

2. Escribe (en pseudocódigo) cómo accedes al puesto de la fila 2, columna 3.

R// estado = salon[2][3]

3. ¿Cuántas casillas tiene en total esta matriz? ¿Cómo lo calculas en general para filas x columnas?

R// Esta matriz tiene: $3 \times 4 = 12$ casillas.
Fórmula general: Total = filas x columnas.

3.2 De 2D a memoria (row-major)

La memoria RAM es una sola fila continua de casillas — no existen "filas y columnas" físicamente en la RAM. Por eso una matriz se guarda "aplanada", fila por fila.

Supón que el salón (3 filas x 4 columnas) se guarda empezando en la dirección 0x1000, con cada valor ocupando 4 bytes, en orden por filas (row-major).

1. Dibuja cómo quedarían las 12 casillas de salón en una sola fila de memoria (aplanadas).

[0][0]	[0][1]	[0][2]	[0][3]	[1][0]	[1][1]	[1][2]	[1][3]	[2][0]	[2][1]	[2][2]	[2][3]
--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------

2. Calcula la dirección de memoria de la casilla [1][2], usando la fórmula:
dirección = base + ((fila × número_de_columnas + columna) × tamaño)

R// **Fórmula:** dirección = base + ((fila x columnas) + columna) x tamaño

Datos: Base = **0x1000 (4096)**, Filas = **3**, Columnas = **4**, Tamaño = **4 bytes**.

dirección = 0x1000 + ((1 x 4) + 2) x 4

dirección = 0x1000 + (4 + 2) x 4 = 0x1000 + 24 bytes = **0x1018**

3. Calcula también la dirección de [0][0] y de [2][3] (la última casilla).

R//

[0][0]: 0x1000 + ((0 x 4) + 0) x 4 = **0x1000**

[2][3]: 0x1000 + ((2 x 4) + 3) x 4 = 0x1000 + (8 + 3) x 4 = 0x1000 + 44 bytes = **0x102C**

3.3 Recorrido con ciclos anidados

Quieres contar cuántos puestos están ocupados (1) en toda la matriz salon.

1. Describe en pseudocódigo el recorrido usando dos ciclos anidados (uno para filas, otro para columnas).

R//

ocupados = 0

Para f desde 0 hasta filas - 1 Hacer:

Para c desde 0 hasta columnas - 1 Hacer:

Si salon[f][c] == 1 Entonces:

ocupados = ocupados + 1

FinSi

FinPara

FinPara

Mostrar ocupados

2. Si la matriz tiene f filas y c columnas, ¿cuál es la complejidad de este recorrido en notación Big O? Explica por qué.

R// $O(n)$ donde n es el número total de casillas.

Hay que revisar cada casilla individualmente exactamente una vez.

3. ¿Qué pasaría con el tiempo de ejecución si f y c se duplican ambos a la vez?

R// Sí las filas se duplican ($2f$) y las columnas se duplican ($2c$), la cantidad total de casillas se multiplica por 4 ($2f \times 2c = 4fc$). Por ende, el tiempo de ejecución **se multiplicará por 4**.

3.4 Conectando todo

En un párrafo corto (5-8 líneas), explica con tus propias palabras la relación entre estos tres conceptos vistos en las últimas tres clases: Big O, arreglos 1D y arreglos 2D. Pista: piensa en por qué entender la complejidad te ayuda a decidir cuándo usar un arreglo 1D, cuándo una matriz, y cuándo buscar de forma lineal o binaria.

R// Aprender sobre Big O es bueno porque nos ayuda a saber qué estructura usar según lo que se necesite hacer. Por ejemplo, en un arreglo normal 1D es muy fácil entrar a una casilla directa si le sabemos la posición, pero si buscamos un dato de uno en uno se puede demorar mucho, a menos que los datos estén organizados y se pueda usar la búsqueda binaria para ir más rápido. Con los arreglos 2D pasa algo parecido: sirven mucho para organizar cosas en filas y columnas, pero si se necesitan revisar todas las casillas ya toca usar dos ciclos unidos y eso hace que se demore más. En resumen, al entender cómo la computadora guarda todo esto en la memoria nos ayuda a usar la mejor opción para que nuestro programa no se ponga lento.